

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	BHUVANESH. S	Reg. No.:	192521004
Section / Batch:	IT	Date:	22/8/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I BHUVANESH. S (192521004) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: BHUVANESH. S

Q5. Dijkstra – Navigation System

A navigation system computes shortest paths between locations in real-time. Analyze how Dijkstra's algorithm can be applied to solve this problem efficiently. Design a solution by explaining step-by-step execution and the use of priority queues. Suggest appropriate data structures and tools for implementation. Analyze time complexity using different approaches. Interpret the performance in real-world navigation systems.

1. Introduction

A navigation system needs to find the **shortest or fastest route** between two locations. The road network can be represented as a **weighted graph**, where locations are vertices and roads are edges.

Dijkstra's Algorithm is a shortest-path algorithm that finds the minimum distance from a source location to other locations when all road weights are non-negative. By using a **priority queue**, the algorithm can efficiently select the next location with the smallest known distance.

In real-world navigation, the edge weight can represent:

- Distance in kilometers
- Travel time
- Fuel cost
- Toll cost
- A combination of multiple factors

2. Modeling the Network as a Graph

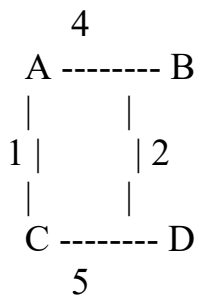
The first step is to convert the navigation network into a graph.

Graph Components

- **Vertices (V):** Represent locations, junctions, cities, or landmarks.
- **Edges (E):** Represent roads connecting two locations.
- **Weights:** Represent distance or estimated travel time.
- **Source:** Starting location.
- **Destination:** Target location.

Example

Consider the following network:



Here:

A, B, C, D → Locations

A-B, A-C, B-D, C-D → Roads

1, 2, 4, 5 → Road weights

If the source is **A** and destination is **D**, Dijkstra's algorithm finds:

A → C → D

Cost = 1 + 5 = 6

3. Problem Statement

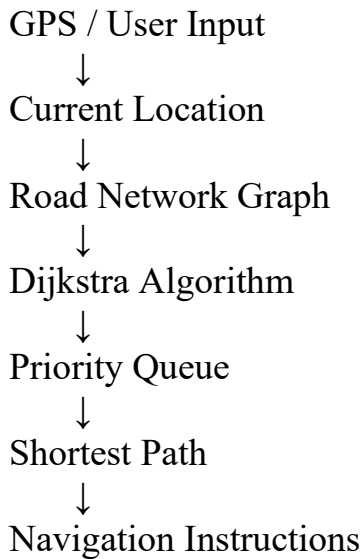
Design a navigation routing system that determines the shortest path between locations using Dijkstra's algorithm.

The system should:

1. Represent locations and roads as a weighted graph.
2. Store the shortest known distance to each location.
3. Use a priority queue to select the nearest unprocessed location.
4. Update distances using edge relaxation.
5. Continue until the destination is reached.
6. Reconstruct and display the shortest route.

4. Designing a Real-Time Routing Optimization Solution

A real-time routing system can be designed using the following components:



Main Components

1. Location Input

The user's current GPS location and destination are obtained.

2. Graph Construction

The road network is represented as a weighted graph.

3. Distance Initialization

Source = 0

All other locations = ∞

4. Priority Queue

A minimum-priority queue stores:

(distance, location)

The location with the smallest distance is processed first.

5. Edge Relaxation

For an edge from u to v:

$\text{new_distance} = \text{distance}[u] + \text{weight}(u,v)$

If:

$\text{new_distance} < \text{distance}[v]$

then update $\text{distance}[v]$.

6. Route Reconstruction

A predecessor array/dictionary stores the previous location, allowing the complete route to be reconstructed.

5. Step-by-Step Execution

Suppose the network is:

```
A --4-- B
|      |
1      2
|      |
C --5-- D
```

Source = A

Step 1: Initialization

$A = 0$

$B = \infty$

$C = \infty$

$D = \infty$

Priority Queue:

$(0, A)$

Step 2: Process A

From A:

$A \rightarrow B = 4$

$A \rightarrow C = 1$

Updated distances:

$$A = 0$$

$$B = 4$$

$$C = 1$$

$$D = \infty$$

Priority Queue:

(1, C)

(4, B)

Step 3: Process C

C has the smallest distance.

$$C \rightarrow D = 1 + 5 = 6$$

Therefore:

$$D = 6$$

Step 4: Process B

$$B \rightarrow D = 4 + 2 = 6$$

This does not improve D because D is already 6.

Step 5: Process D

D is reached with minimum cost.

Final Result

Shortest Path = $A \rightarrow C \rightarrow D$

Minimum Cost = 6

6. Use of Priority Queue

A priority queue is important for improving Dijkstra's efficiency.

Instead of checking every unvisited location, it always provides the location with the **smallest current distance**.

Example:

Priority Queue

(1, C)
(4, B)
(6, D)

The smallest value (1, C) is removed first.

Advantages

- Faster minimum-distance selection
- Efficient for large graphs
- Reduces unnecessary processing
- Suitable for sparse road networks
- Can be implemented using a **binary min-heap**

7. Appropriate Data Structures and Tools

Requirement	Recommended Structure
Road network	Graph
Graph representation	Adjacency List
Shortest distances	Array / Dictionary
Previous locations	Array / Dictionary
Visited locations	Set
Minimum selection	Priority Queue / Min-Heap
GPS location	Coordinates
Implementation	Python / C++ / Java

Recommended Combination

Adjacency List + Min-Heap Priority Queue

This combination is particularly suitable for large, sparse road networks.

8. Scalability for Large Networks

Real navigation systems can contain **millions of locations and roads**. Running basic Dijkstra repeatedly can become expensive.

Techniques for scalability

1. Adjacency List

Stores only existing road connections instead of a complete matrix.

2. Priority Queue

Efficiently selects the next closest location.

3. Bidirectional Search

Searches from both the source and destination and can reduce the search area.

4. A* Algorithm

Uses a heuristic, such as straight-line distance, to guide the search toward the destination.

5. Hierarchical Routing

Divides roads into levels such as:

Local Roads



Main Roads



Highways

This reduces unnecessary exploration.

6. Preprocessing

Frequently used routes and road information can be preprocessed or cached to reduce computation time.

9. Time Complexity and Memory Usage

Let:

- V = number of vertices/locations
- E = number of edges/roads

Approach 1: Adjacency Matrix

Using a simple array:

Time Complexity:

Space Complexity:

This is suitable for small or dense graphs.

Approach 2: Adjacency List + Binary Heap

Time Complexity:

Often simplified to:

for connected sparse graphs.

Space Complexity:

This is generally more suitable for large road networks.

Approach 3: Fibonacci Heap

The theoretical complexity is:

However, Fibonacci heaps are more complicated and are not always the most practical choice.

10. Performance in High-Speed Communication Systems

In systems where routing decisions must be made quickly, the algorithm should minimize computation and memory usage.

Dijkstra can provide reliable shortest paths, but repeatedly running it on a huge network may introduce latency.

Performance Improvements

- Use priority queues.
- Use adjacency lists.
- Process only relevant parts of the network.
- Cache frequently requested routes.
- Use parallel or distributed processing where appropriate.
- Use A* or hierarchical routing for very large networks.

For a navigation system, **fast response time is important** because traffic conditions and user locations can change continuously.

11. Handling Real-Time Traffic

Basic Dijkstra assumes that edge weights are known and non-negative.

In a real navigation system, road weights can change because of:

- Traffic congestion
- Accidents
- Road closures
- Construction
- Weather
- Temporary speed restrictions

For example:

Normal Traffic:

$A \rightarrow B = 10$ minutes

Heavy Traffic:

$A \rightarrow B = 25$ minutes

The system can update the edge weight and recalculate the route.

Thus, Dijkstra can be used as a foundation, but practical navigation systems usually require additional mechanisms for **dynamic weights and large-scale routing**.

12. Comparison with Real Life

Dijkstra Concept	Real-Life Navigation
Vertex	Location / Junction
Edge	Road
Weight	Distance / Travel time
Source	Current GPS location
Destination	User's selected location
Priority Queue	Chooses nearest candidate
Shortest Path	Best route
Weight Update	Traffic changes
Graph	Complete road network

Example

If two routes are available:

Route 1: 10 km but heavy traffic

Route 2: 12 km but low traffic

A navigation system using **travel time as the edge weight** may select Route 2 because it provides a shorter travel time.

13. Advantages

- Finds shortest paths reliably.
- Easy to understand and implement.
- Works well with non-negative edge weights.
- Priority queues improve efficiency.
- Suitable for many graph-based routing problems.
- Can use distance or travel time as weights.

14. Limitations

- Does not directly handle negative edge weights.
- Basic Dijkstra can explore many unnecessary locations.
- Recalculation can be expensive for very large maps.
- Dynamic traffic requires updated edge weights.
- For large real-world maps, specialized routing algorithms can be faster.

15. Suitable Tools for Implementation

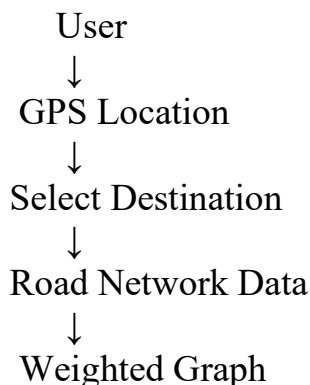
Software

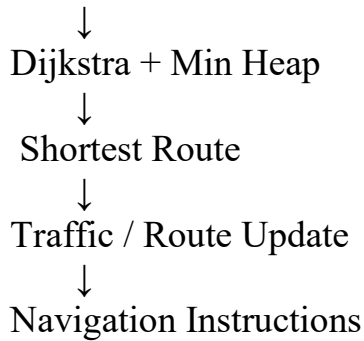
- **Python** – easy implementation using heapq
- **C++** – efficient using priority_queue
- **Java** – using PriorityQueue

Supporting Technologies

- GPS for current location
- Map/road databases for road networks
- Graph data structures
- Priority queues
- Real-time traffic data
- Caching systems

16. Overall System Architecture





17. Conclusion

Dijkstra's algorithm is an important shortest-path technique for navigation systems. By representing **locations as vertices and roads as weighted edges**, the algorithm can determine the minimum-cost route. Using an **adjacency list and priority queue** provides good performance for large sparse graphs, with a typical time complexity of $O((V + E) \log V)$ and space complexity of $O(V + E)$.

However, real-world navigation systems must also consider **traffic, road closures, changing travel times, and extremely large networks**. Therefore, Dijkstra is an excellent fundamental algorithm, while practical systems may combine it with **A*, bidirectional search, hierarchical routing, preprocessing, and real-time traffic updates**.

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

